

Notes on Compiler Construction - Overview

1. Introduction to Compilers and Interpreters

The Fundamental Problem

How do we execute high-level programming code?

Example Code:

```
int nPos = 0;

int k = 0;

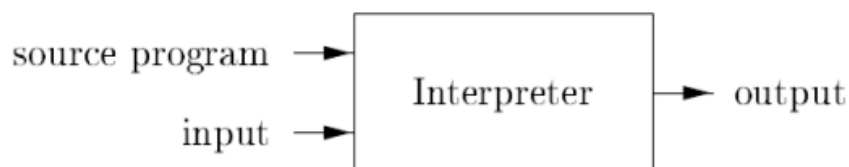
while (k < length) {
    if (a[k] > 0) {
        nPos++;
    }
}
```

The computer's hardware can only execute machine code (binary instructions). We need a way to bridge the gap between human-readable code and machine-executable instructions.

Two Approaches: Interpreters vs Compilers

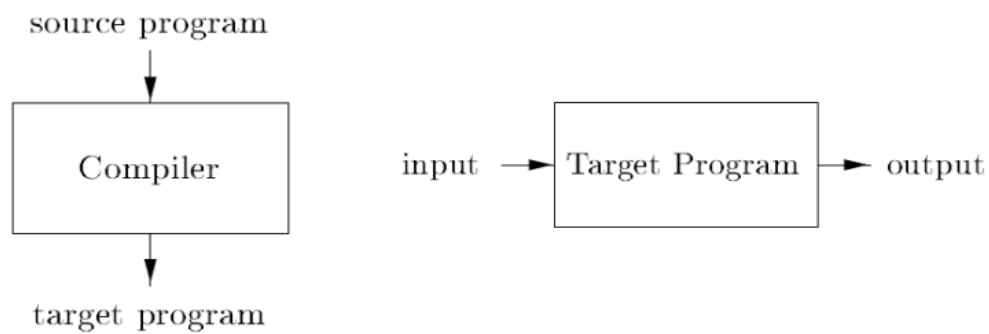
Interpreter:

- A program that reads a source program and produces the results of executing that program
- Directly executes the source code without producing a separate executable
- Execution and analysis are **interleaved**



Compiler:

- A program that translates a program from one language (source) to another (target)
- Produces an executable that can be run separately
- Analysis happens **before** execution



2. Common Issues for Both

Understanding the Input

Both compilers and interpreters must read and "understand" the input program, which is initially just a stream of characters.

Example Character Stream:

```
w h i l e ( k < l e n g t h ) { <newline> <tab> i f ( a [ k ] > 0 ) <newline> <tab> <tab> { n P o s + +  
; } <newline> <tab> }
```

This stream needs to be:

1. Broken into meaningful units (tokens)
2. Organized into structures (syntax)
3. Checked for correctness (semantics)
4. Translated or executed

3. Interpreter Architecture

How Interpreters Work

Basic Structure:

```
running = true;  
while (running) {  
    analyze next statement;  
    execute that statement;  
}
```

Characteristics of Interpreters

Advantages:

- **Immediate execution:** No separate compilation step
- **Good for debugging:** Can stop and inspect state at any point
- **Interactive:** Can modify code and see results immediately
- **Portability:** Same interpreter runs on different platforms

Disadvantages:

- **Repeated analysis:** Statements are analyzed every time they execute
- **Slower execution:** Especially problematic in loops and frequently called functions
- **Memory overhead:** Need to keep source code and interpreter in memory

Example of Repeated Analysis:

```
for i in range(1000000):
```

```
    x = x + 1 # This line is analyzed 1,000,000 times!
```

Common Interpreted Languages

- **Scripting languages:** Python, Ruby, Perl, JavaScript
- **Shell languages:** bash, awk, sed
- **Functional languages:** Scheme, Lisp, ML (historically)
- **Document languages:** PostScript, PDF interpreters
- **Virtual machines:** Java VM (interprets bytecode)

When Interpreters Work Well

Interpreters are particularly effective when:

- Interpreter overhead is low relative to statement execution cost
- Development speed is more important than execution speed
- Dynamic features are needed (eval, reflection)
- Portability across platforms is critical

4. Compiler Architecture

How Compilers Work

Process:

1. **Read and analyze** entire program
2. **Translate** to semantically equivalent program in another language
3. **Optimize** to improve performance
4. **Generate** executable code

Characteristics of Compilers

Advantages:

- **Faster execution:** Code is translated once, runs many times
- **Optimization opportunities:** Can analyze entire program for improvements
- **Error detection:** Many errors caught before execution
- **No source code needed at runtime:** Only executable is distributed

Disadvantages:

- **Compile-time overhead:** Must compile before running
- **Edit-compile-run cycle:** Slower development iteration
- **Complex toolchain:** Multiple phases and tools needed

The Compilation Tradeoff

Tradeoff: Compile-time overhead (preprocessing step) vs execution performance

There is a tradeoff between compile-time overhead and execution performance: spending more time preprocessing and optimizing during compilation results in faster and more efficient execution.

When to use compilers:

- Performance-critical applications
- Production code that runs frequently
- Large-scale systems
- When distributing to end users (hide source code)

Common Compiled Languages

- **Systems languages:** C, C++, Rust
- **Scientific computing:** FORTRAN

- **Enterprise:** Java (to bytecode), C#
- **Legacy:** COBOL, Pascal

5. Hybrid Approaches

The Best of Both Worlds

Many modern languages use a combination of compilation and interpretation.

Java: A Well-Known Example

Step 1: Compilation

Java Source (.java) → Java Compiler → Bytecode (.class files)

Step 2: Execution (Two Options)

Option A: Interpretation

- Java Virtual Machine (JVM) interprets bytecode directly
- Slower but portable

Option B: Just-In-Time (JIT) Compilation

- JVM detects "hot spots" (frequently executed code)
- Compiles it to machine code while running
- **Adaptive optimization:** Can optimize based on runtime behavior
- Standard approach in modern JVMs

Advantages of Hybrid Approach

1. **Portability:** Bytecode runs on any platform with a JVM
2. **Fast startup:** Begin execution immediately
3. **High performance:** Hot code runs at native speed
4. **Dynamic optimization:** Optimize based on actual usage patterns

Other Examples

- **C#/.NET:** Similar to Java (MSIL bytecode)
- **Python:** Often compiled to bytecode (.pyc files) then interpreted
- **JavaScript:** Modern engines use JIT compilation (V8, SpiderMonkey)

6. Phases of a Compiler

A compiler is organized into several distinct phases, each with a specific responsibility.

Source Code

↓

[Lexical Analyzer (Scanner)]

↓

Tokens

↓

[Syntax Analyzer (Parser)]

↓

Parse Tree / AST

↓

[Semantic Analyzer]

↓

Annotated AST

↓

[Intermediate Code Generator]

↓

Intermediate Code

↓

[Optimizer]

↓

Optimized Intermediate Code

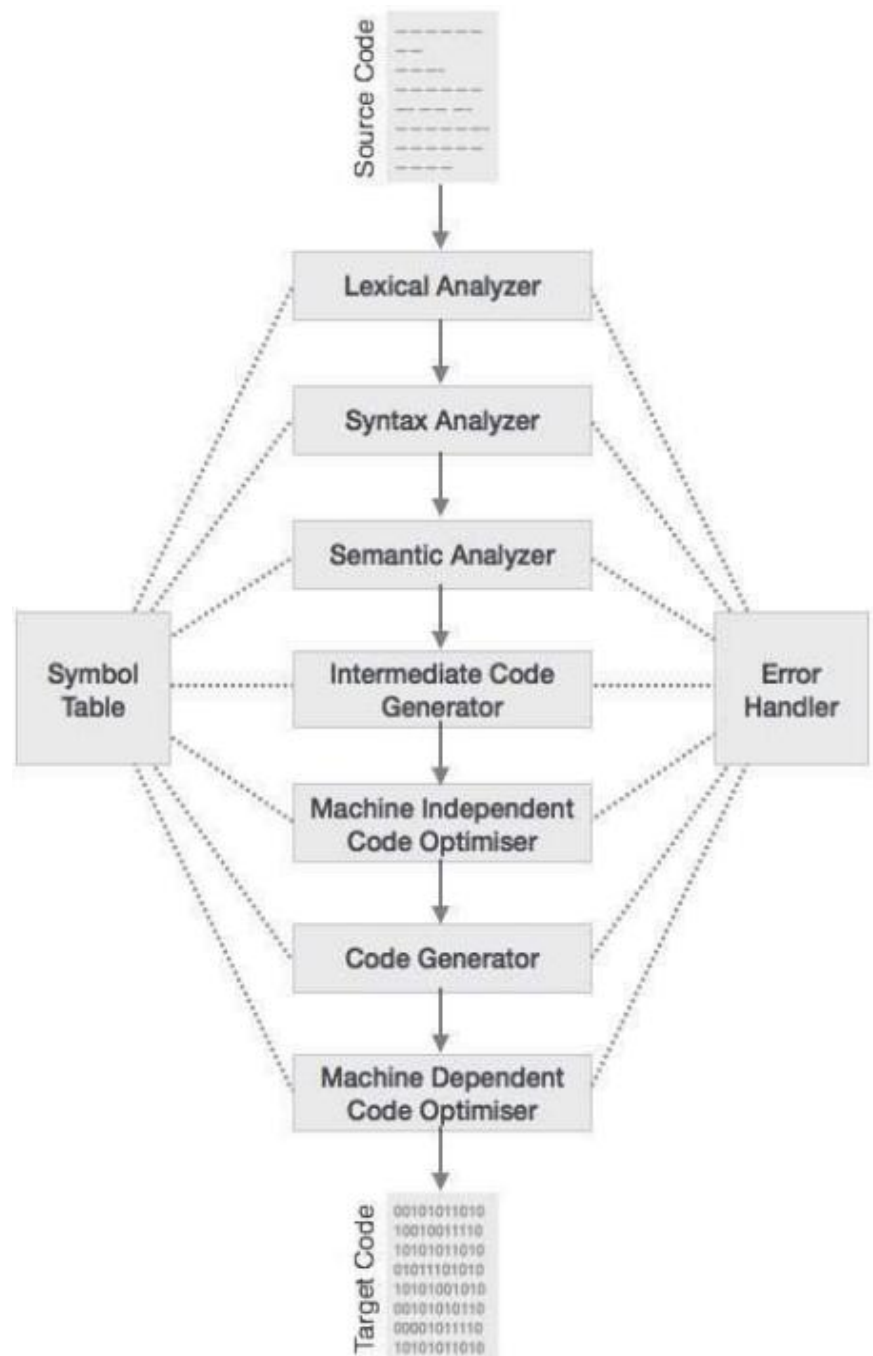
↓

[Code Generator]

↓

Target Machine Code

Each phase uses information stored in the **Symbol Table**.



7. Lexical Analysis (Scanner)

The scanner reads the source program as a stream of characters and groups them into meaningful units called **tokens**.

Key Responsibilities

1. **Group characters into lexemes**
 - A lexeme is a sequence of characters that forms a meaningful unit
 - Example: "while", "position", "123", ">=", ";"
2. **Remove whitespace and comments**
 - These are not needed for compilation
 - Simplifies later phases
3. **Identify token types**
 - Each lexeme is classified as a specific token type
 - Example: "while" → KEYWORD, "123" → NUMBER, "x" → IDENTIFIER
4. **Detect lexical errors**
 - Invalid characters (like # in Java)
 - Malformed numbers or strings

Token Structure

Token: <token_name, attribute_value>

- **token_name:** Used by the parser (e.g., IDENTIFIER, NUMBER, KEYWORD)
- **attribute_value:** Points to symbol table entry with additional information

Example: Lexemes and Tokens

Lexeme:	;	=	index	tmp	21	64.32
Token:	SEMI-COLON	ASSIGN	ID	ID	NUM_INT	NUM-FLOAT

Example:

position = initial + rate * 60

↓

Lexical Analyzer

↓

<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE

Why Lexical Analysis is Needed

1. **Simplifies parsing:** Parser works with tokens, not characters
2. **Makes compiler faster:** Scanning is simpler than parsing
3. **Separates concerns:** Tokenizing is separate from syntax analysis
4. **Early error detection:** Catches invalid characters immediately
5. **Regular expressions:** Token patterns can be specified using regex

Regular Expressions in Scanning

Tokens are typically defined using regular expressions:

IDENTIFIER = [a-zA-Z][a-zA-Z0-9]*

INTEGER = [0-9]+

FLOAT = [0-9]+\.[0-9]+

KEYWORD = if|while|for|return|...

OPERATOR = +|-|*|/|==|!=|<=|>=|...

8. Syntax Analysis (Parser)

The parser groups tokens into "grammatical phrases" to discover the underlying structure of the source program.

Key Responsibilities

1. **Build program structure**
 - Groups tokens according to language grammar
 - Creates a hierarchical representation (parse tree or AST)
2. **Find syntax errors**
 - Detects invalid token sequences
 - Reports meaningful error messages
3. **Check static semantics (sometimes)**
 - Use of undeclared variables
 - Multiple declarations of same variable
4. **Generate intermediate representation**
 - Parse tree (full detail)
 - Abstract Syntax Tree (AST) (simplified, more useful)

Example: Syntax Error Detection

Source Code:

```
position = 5; *
```

Token Sequence:

IDENTIFIER ASSIGN_OP INTEGER SEMICOLON MULT_OP

Problem: All tokens are valid individually, but this sequence is grammatically incorrect (what follows the semicolon?).

Parse Tree vs Abstract Syntax Tree

Parse Tree:

- Contains all grammatical details
- Includes every rule used in derivation
- Larger, more detailed

Abstract Syntax Tree (AST):

- Simplified version of parse tree
- Removes unnecessary grammatical details
- Focuses on program meaning
- More compact and easier to work with

Example: Building an AST

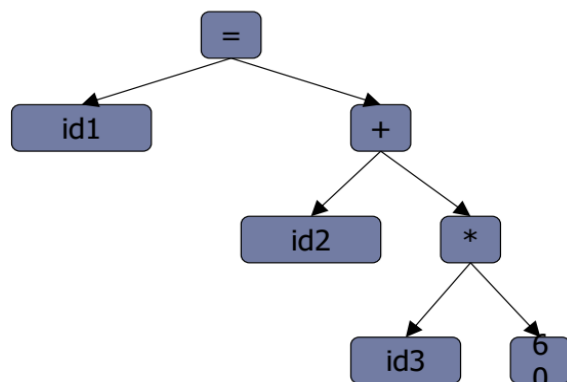
Source Code:

```
id1 = id2 + id3 * 60
```

Token Sequence:

<id1, 1> <=> <id2, 2> <+> <id3, 3> <*> <60>

Abstract Syntax Tree:



Structure:

- Interior nodes are **operators**
- Children of a node are **operands**
- Reflects operator precedence (multiplication before addition)
- Left-to-right ordering for operands

Context-Free Grammars

Parsers use **Context-Free Grammars (CFG)** to define syntax:

statement $\rightarrow$ assignment | if-stmt | while-stmt | ...

assignment $\rightarrow$ IDENTIFIER ASSIGN expr SEMICOLON

expr $\rightarrow$ expr PLUS term | term

term $\rightarrow$ term MULT factor | factor

factor $\rightarrow$ IDENTIFIER | NUMBER | LPAREN expr RPAREN

9. Semantic Analysis

The semantic analyzer uses the syntax tree to check for additional correctness properties that cannot be verified by syntax alone.

Key Responsibilities

1. **Type checking**
 - Ensure operations are performed on compatible types
 - Example: Cannot add a string to an integer (in many languages)
2. **Type conversion/coercion**
 - Automatically convert between compatible types
 - Example: int to float when needed
3. **Variable declaration checking**
 - Variables must be declared before use
 - Variables cannot be multiply declared in same scope
4. **Function call checking**
 - Correct number of arguments
 - Correct types of arguments

5. Array usage checking

- Array variables must use array syntax
- Array indices must be integers

6. L-value checking

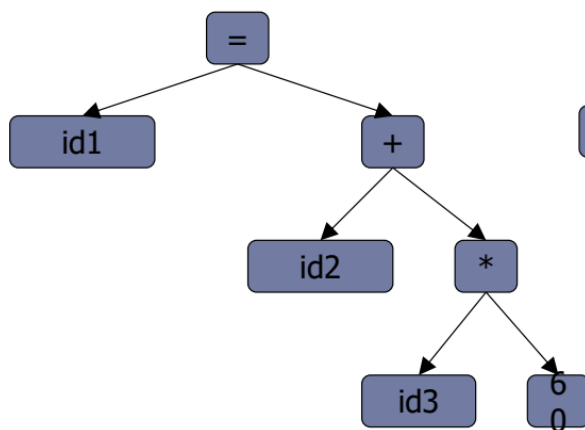
- Left side of assignment must be a variable (L-value)
- Cannot assign to constants or expressions

7. Scope checking

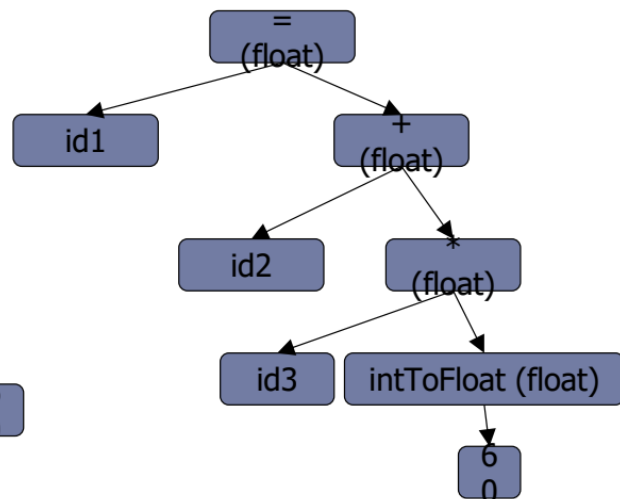
- Names resolved correctly in nested scopes

Example: Type Checking and Coercion

Abstract-Syntax Tree:



Annotated Abstract-Syntax Tree:



Changes made:

1. Added type annotations to operators
2. Inserted **intToFloat** conversion for the integer literal 60
3. Verified all operations are type-correct

Static vs Dynamic Semantics

Static Semantics:

- Checked at compile time
- Type checking, declaration checking
- Can prevent many errors before execution

Dynamic Semantics:

- Checked at runtime
- Array bounds checking, null pointer checks
- Cannot always be verified statically

10. Intermediate Code Generation

Translate the abstract syntax tree into a low-level, machine-like intermediate representation.

Characteristics of Good Intermediate Code

1. **Easy to produce** from AST
2. **Easy to translate** to machine code
3. **Machine independent** (portable)
4. **Facilitates optimization**

Three-Address Code

A common intermediate representation where each instruction has **at most three operands**.

Format:

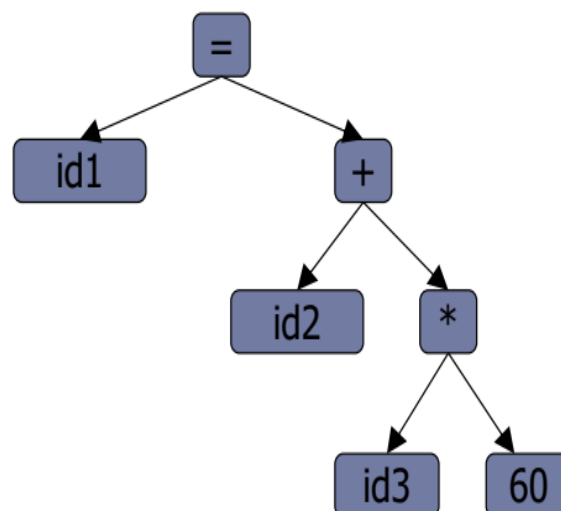
result = operand1 operator operand2

Example: Three-Address Code Generation

Source Code:

id1 = id2 + id3 * 60

AST:



Three-Address Code:

temp1 = intToFloat(60)

temp2 = id3 * temp1

temp3 = id2 + temp2

id1 = temp3

Another Example**Source Code:**

x = (a + b) * (c - d);

Intermediate Code:

t1 = a + b

t2 = c - d

t3 = t1 * t2

x = t3

Why Intermediate Code is Needed**1. Portability**

- The same compiler front-end can be used for many machines
- Only the final code generator needs to change for a new computer

2. Optimization

- Many optimizations are easier on intermediate code
- Example: remove useless code, simplify calculations, reuse results

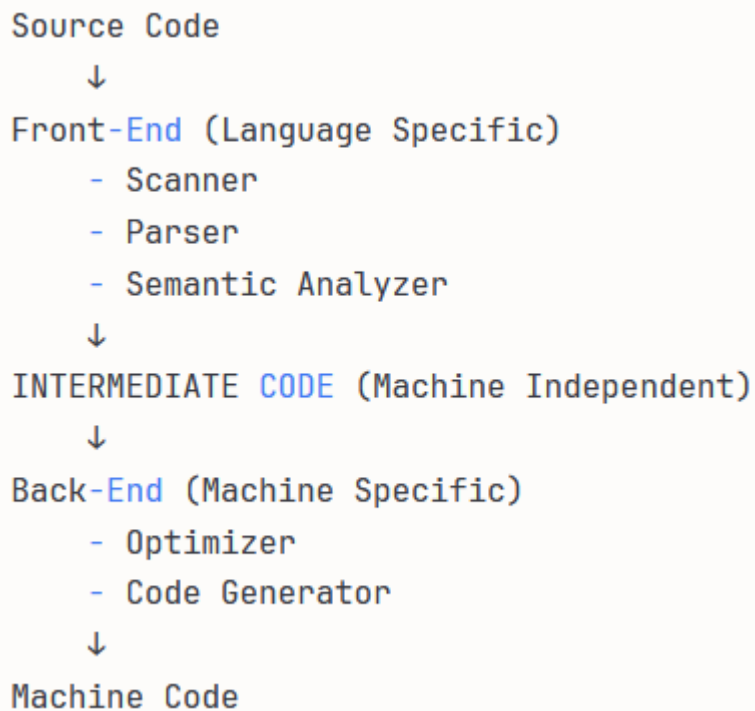
3. Simpler Code Generation

- Breaks complex high-level code into simple steps
- Makes it easier to convert into machine instructions

4. Decoupling (Separation of Work)

- Front-end: understands the programming language
- Back-end: knows the machine hardware
- Intermediate code: connects the front-end and back-end

Compiler Architecture with Intermediate Code



This architecture allows:

- **N languages × M machines** with only **N + M** components
- Without intermediate code: need **N × M** complete compilers!

11. Code Optimization

Purpose

Improve the intermediate code to make the resulting program:

- **Faster** (run more quickly)
- **Smaller** (use less memory)
- **More energy efficient** (use less power)

Example Optimization

Before Optimization:

```
temp1 = intToFloat(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

After Optimization:

```
temp1 = id3 * 60.0    // Constant folded  
id1 = id2 + temp1     // Eliminated temp3
```

Improvements:

1. **Constant folding:** `intToFloat(60)` computed at compile time → 60.0
2. **Copy propagation:** Eliminated unnecessary `temp3`
3. Reduced from 4 to 2 instructions

Common Optimization Techniques

1. Constant Folding

Before: `x = 2 + 3`

After: `x = 5`

2. Constant Propagation

Before: `x = 5; y = x + 2`

After: `x = 5; y = 7`

3. Dead Code Elimination

Before: `x = 5; y = 10; z = x + 3` // `y` is never used

After: `x = 5; z = x + 3`

4. Common Subexpression Elimination

Before: `a = b + c; d = b + c`

After: `temp = b + c; a = temp; d = temp`

5. Loop Optimization

- **Loop-invariant code motion:** Move calculations outside loops
- **Strength reduction:** Replace expensive operations with cheaper ones

Before:

```
for (i = 0; i < n; i++)
```

```
    x = y * 2; // y * 2 doesn't change in loop
```

After:

```
temp = y * 2;
for (i = 0; i < n; i++)
    x = temp;
```

6. Inlining

- Replace function calls with function body
- Eliminates call overhead

12. Code Generation

Purpose

Generate target machine code from the (optimized) intermediate code.

Key Responsibilities

1. **Instruction selection**
 - Choose appropriate machine instructions
 - Handle instruction set limitations
2. **Register allocation**
 - Assign variables to machine registers
3. **Instruction scheduling**
 - Order instructions for best performance

Example: Code Generation

Optimized Intermediate Code:

```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

Generated Assembly Code:

```
LDF  R2, id3      ; Load id3 into register R2
MULF R2, R2, #60.0 ; Multiply R2 by 60.0
LDF  R1, id2      ; Load id2 into register R1
ADDF R1, R1, R2    ; Add R2 to R1
STF  id1, R1      ; Store R1 to id1
```

13. Symbol Tables

Purpose

The symbol table keeps track of all names (identifiers) declared in the program.

What Information is Stored

For each identifier, store:

- **Name:** The identifier string
- **Type:** Data type (int, float, array, function, etc.)
- **Scope:** Where the identifier is valid
- **Location:** Memory address or register
- **Additional attributes:** Array dimensions, function parameters, etc.

Scope Management

Scope: The region of code where an identifier is valid.

Example:

```
int x = 5;    // Global scope
```

```
void foo() {  
    int y = 10;  // Function scope  
    {  
        int z = 15; // Block scope  
    }  
    // z not valid here  
}  
// y not valid here
```

Uses of Symbol Tables

During Semantic Analysis:

1. Check if variables are declared before use
2. Check for multiple declarations in same scope

3. Verify parameter types match function declaration
4. Type checking

During Code Generation:

1. Determine memory locations for variables
2. Resolve references to variables

14. Compiler Modularity and Architecture

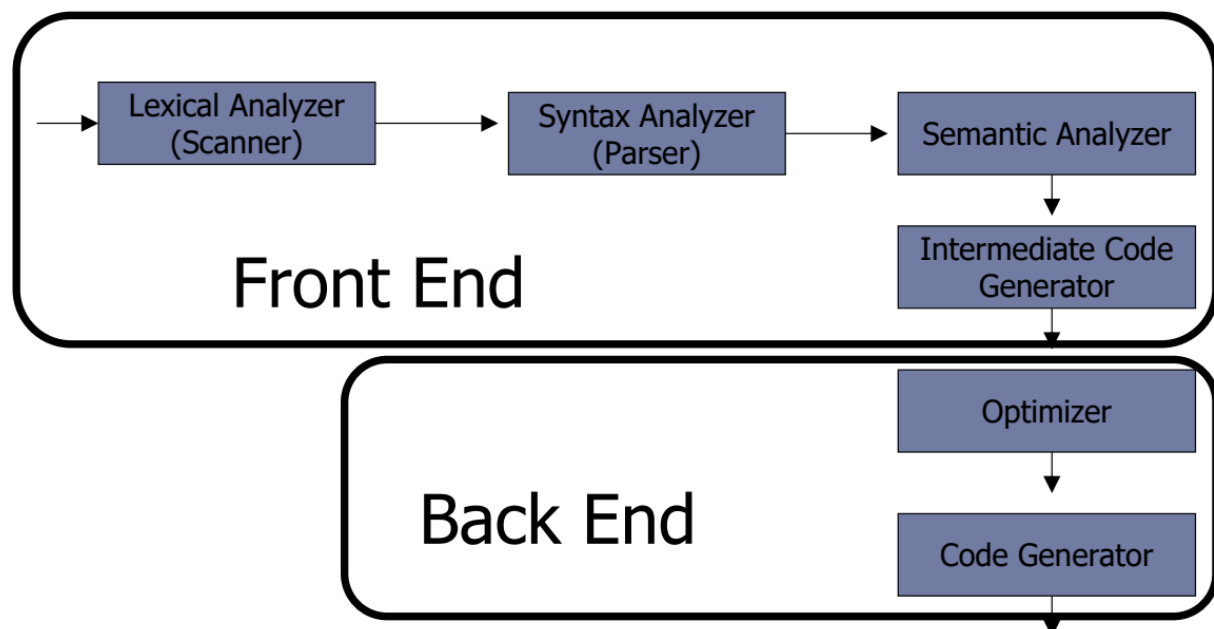
Front-End vs Back-End

Front-End (Language Dependent):

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code Generation

Back-End (Machine Dependent):

- Optimization
- Code Generation



Benefit: Separation of concerns allows:

- One front-end, multiple back-ends (support many architectures)
- Multiple front-ends, one back-end (support many languages)

Many-to-Many Compilation

Without Intermediate Representation:

$N \text{ languages} \times M \text{ machines} = N \times M \text{ compilers needed}$

With Intermediate Representation:

$N \text{ front-ends} + M \text{ back-ends} = N + M \text{ components}$

Example:

- 4 languages (Java, C, C++, Fortran)
- 3 machines (x86, ARM, RISC-V)
- Without IR: $4 \times 3 = 12$ complete compilers
- With IR: $4 + 3 = 7$ components

Diagram: Compiler Modularity

Saw slide

Why Study Compiler**1. Become a Better Programmer**

- Understand how high-level code is translated into machine code
- Write more efficient and optimized programs
- Gain better debugging skills by knowing what happens behind the scenes

2. Compiler Concepts Are Used Everywhere

- Applied in parsing, databases (query optimization), text processing, hardware design, AI systems, and mathematical software

3. Combination of Theory and Practice

- Based on formal theory like automata, grammars, and type systems
- Also deals with practical issues such as optimization, error handling, and performance

4. Uses Knowledge from Many CS Areas

- Algorithms, artificial intelligence techniques, operating systems, and computer architecture

5. Practical and Career-Relevant

- Even without writing a full compiler, you will build parsers, interpreters, DSLs, and code generators in real projects